claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_c5395511-084\agent-a14369f296d36c0f0.jsonl`
- SHA-256: `9057e3a0089c5790ca4ab2b3d26f5ccb852a9e2be110aaa0c2c168a74fd0d65e`
- Source modified: `2026-07-06T21:33:07+00:00`
- Imported at: `2026-07-08T16:00:09+00:00`
- project: `wf_c5395511-084`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T21:25:50.738Z)

```
Reviewing GitHub PR #51 "feat(policy): add F2 topic preferences" (branch feat/f2-topic-
preferences -> main). ADR 0012 F2.
+417/-16, 8 files. A checked-out copy is at C:/Users/avidu/Projects/_wt-pr51 (read files there,
or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f2-topic-preferences:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>  (db._apply_schema on
aiosqlite ":memory:").

WHAT F2 DOES:
- models.py: VideoRecord gains a 'text' field (caption). db.py: insert_video /
_content_item_to_video /
  get_video_by_content_fingerprint / list_videos_for_user propagate text into/out of
content_items.text (which existed since ADR 0011).
- policy.py: TopicPreferences + parse_topic_preferences + load_topic_preferences
(json.loads(policy.config_json)); TopicPreferenceStage
  (Tier 2): exclude-veto, include any/all, casefold substring match over source.title +
content_item.text.
- telethon_client.py: _message_text(message) (message/raw_text, strip, None if empty);
_content_item_preview(...) builds a throwaway
  ContentItem (incl. a computed dedup_key) for topic eval; _evaluate_topic_preferences(conn,
account, source, item) loads the default
  policy and runs TopicPreferenceStage. on_new_message (and backfill.py) now:
matched=engine.evaluate(); if matched and source:
  run topic stage, matched=topic_result.passed; then insert VideoRecord(text=...) and deliver if
matched.
- The live handler on_new_message and backfill_user are '# pragma: no cover'.

VERIFIED by the main reviewer already: caption text reaches content_items.text; topics survive a
/set (F1's merge) and are read via
the RAW config (not FilterConfig); topic gating passes all-includes / vetoes excludes. Gate:
ruff clean, 513 passed @ 95.58%.
Your job: find bugs/regressions/edge cases in the WIRING and stage the happy path missed.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No
praise. Concrete failure scenario.
Rank most-severe first. Severity: high (real bug/regression/wrong delivery decision), medium
(edge/latent/design-inconsistency that will bite), low (hygiene/scope-note).

DIMENSION: live-path + backfill wiring correctness (telethon_client.on_new_message,
backfill.backfill_user).
Investigate:
- Short-circuit/AND semantics: topic stage runs only 'if matched and source is not None' and
sets matched=topic_result.passed. Confirm it
```

```
  can only turn a metadata-matched item False (never False->True), and that delivery is
correctly gated on the final matched. Any path where
  a topic-rejected item still gets delivered, or a topic-accepted item skipped?
- source resolution: on_new_message fetches source before AND after upsert_channel; for a
watched channel (post should_ingest_source gate) is
  source always non-None when topics run? If source is None, topics are SKIPPED (item delivered
on metadata alone) ? is that correct/intended?
- backfill parity: backfill now calls upsert_channel (it didn't before) to get source; does that
clobber a user's watched flag (should be safe
  post-#41)? Does backfill gate delivery on the topic result the same way as live? Any
divergence between the two wirings?
- The 'matched' passed into _content_item_preview and stored on the throwaway ContentItem ? does
it affect anything? insert uses VideoRecord.matched
  which now = post-topic matched; confirm consistent.
- reasons/logging: topic_result.reason appended to reasons ? any None/attr issues.
Run repros where feasible (you can call db + policy directly; the pragma:no-cover handler body
can be reasoned about). Report findings.
```

2. assistant (2026-07-06T21:25:53.517Z)

```
I'll review the live-path and backfill wiring for PR #51. Let me start by reading the relevant
files.
```

3. user (2026-07-06T21:25:53.932Z)

```
File does not exist. Note: your current working directory is C:\Users\avidu\Projects\Telegram.
```

4. assistant (2026-07-06T21:25:57.663Z)

```
The worktree path uses a different naming. Let me find the correct paths.
```

5. user (2026-07-06T21:25:59.843Z)

```
tg_video_filter
---
docker-compose.yml
Dockerfile
docs
pyproject.toml
README.md
scripts
src
tests
```

6. user (2026-07-06T21:26:05.608Z)

```
__init__.py
__main__.py
__pycache__
backfill.py
bootstrap.py
command_service.py
commands.py
config.py
db.py
delivery.py
filters
manager.py
models.py
policy.py
telethon_client.py
```

7. user (2026-07-06T21:26:07.740Z)

```
      """Telethon ingestion: client factory, event handler, and metadata extraction.

      Architecture (see docs/design.md §5):

          NewMessage event
            -> extract_video_meta(message)   # pure, unit-tested
            -> dedup via db.get_video()
            -> FilterEngine.evaluate(meta)
            -> db.insert_video(...) + delivery.forward_to_target(...,
target=cfg.delivery_target)

      The metadata extraction is deliberately a **pure** function taking a Telethon
      ``Message`` and returning a ``VideoMeta | None``. This isolates the only piece
      of logic that's worth unit-testing (everything else is Telethon/async glue).
      """

      from __future__ import annotations

      import logging
      import sqlite3
      from pathlib import Path
      from typing import Protocol

      import aiosqlite
      from telethon import TelegramClient
      from telethon.events import NewMessage
      from telethon.tl.types import (
          Document,
          DocumentAttributeAnimated,
          DocumentAttributeVideo,
          Message,
          MessageMediaDocument,
      )

      from tg_video_filter import db
      from tg_video_filter.commands import ENGINE_KEY
      from tg_video_filter.delivery import build_link_text, deliver
      from tg_video_filter.filters import FilterEngine, VideoMeta
      from tg_video_filter.models import (
          Channel,
          ContentItem,
          DeliveryMode,
          Source,
          TelegramSession,
          User,
          VideoRecord,
      )
      from tg_video_filter.policy import PolicyContext, StageResult, TopicPreferenceStage

      log = logging.getLogger(__name__)


      # --- pure metadata extraction ------------------------------------------------


      def extract_video_meta(message: Message) -> VideoMeta | None:
          """Extract ``VideoMeta`` from a Telethon ``Message``, or ``None`` if not a video.

          Returns ``None`` when the message has no video/document media. Returns a
          ``VideoMeta`` (with fields possibly ``None``) for a video whose individual
          attributes are missing ? the filter engine decides how to treat gaps.

          Telegram stores video metadata in two places:
            - ``Document``: ``size`` (bytes), ``mime_type`` (str), ``attributes`` (list)
            - ``DocumentAttributeVideo`` (inside ``attributes``): ``duration`` (float s),
```

```
            ``w`` (width), ``h`` (height)

        Note: we inspect ``message.media`` directly rather than the ``.video``
        convenience property, because the latter only exists on Telethon's *custom*
        Message wrapper (applied at runtime) and is ``None`` on the raw TL Message
        used in tests. A Document is considered a video iff it contains a
        ``DocumentAttributeVideo``.
        """
        media = getattr(message, "media", None)

        # Real Telegram messages wrap documents in MessageMediaDocument.
        # Unwrap it to get the actual Document.
        if isinstance(media, MessageMediaDocument):
            media = media.document

        if not isinstance(media, Document):
            return None

        # Only treat this as a video if the document carries a video attribute.
        has_video_attr = any(isinstance(a, DocumentAttributeVideo) for a in
media.attributes)
        if not has_video_attr:
            return None

        meta = VideoMeta(
            size_bytes=media.size,
            mime_type=media.mime_type or None,
        )

        # Find the video attribute for duration/dimensions and round/nosound flags.
        video_update: dict = {}
        for attr in media.attributes:
            if isinstance(attr, DocumentAttributeVideo):
                if attr.duration:
                    video_update["duration_s"] = int(round(attr.duration))
                if attr.w:
                    video_update["width"] = attr.w
                if attr.h:
                    video_update["height"] = attr.h
                video_update["is_round"] = attr.round_message  # may be None
                video_update["nosound"] = attr.nosound  # may be None
                break

        # Check for GIF (DocumentAttributeAnimated present).
        is_gif = any(isinstance(a, DocumentAttributeAnimated) for a in media.attributes)

        meta = meta.model_copy(update={**video_update, "is_gif": is_gif})
        return meta


    # --- client factory -----------------------------------------------------


    def build_client(
        session: TelegramSession, settings: SettingsLike
    ) -> TelegramClient:  # pragma: no cover
        """Build a Telethon client with a file-backed SQLiteSession (ADR 0010).

        The session path is read from the ``telegram_sessions.session_path`` column
        (data-driven, no longer derived from the PK at runtime) and resolved
        against ``settings.sessions_dir``. The directory is created if missing.
        """
        sessions_dir = Path(settings.sessions_dir)
        sessions_dir.mkdir(parents=True, exist_ok=True)
        # session_path is stored as a basename (e.g. "user_111.session"); join with
```

```
        # the runtime sessions_dir so the migration did not need to know the path.
        session_path = sessions_dir / session.session_path

        client = TelegramClient(
            str(session_path),
            api_id=settings.tg_api_id,
            api_hash=settings.tg_api_hash,
        )
        return client


    class SettingsLike(Protocol):
        """Structural type for the subset of Settings the client factory needs.

        Using a Protocol keeps the module importable in tests without dragging in
        the real Settings class. At runtime ``config.Settings`` satisfies this shape.
        """

        tg_api_id: int
        tg_api_hash: str
        sessions_dir: str


    # --- event handler wiring ---------------------------------------------------


    def make_new_message_handler(
        *,
        user: User,
        conn: aiosqlite.Connection,
        engine_holder: dict[str, FilterEngine],
        client: TelegramClient,
        delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
        delivery_target: str = "me",
    ) -> _NewMessageCallback:
        """Return an async callback for ``events.NewMessage``.

        The callback:
          1. extracts VideoMeta + document_id (skips if not a video),
          2. dedups via two-tier content dedup (document.id, then metadata fingerprint),
             plus the existing (channel, message) instance dedup,
          3. runs the filter engine (read from ``engine_holder`` so config
             changes via ``/set`` take effect immediately),
          4. inserts a VideoRecord row,
          5. delivers a match to ``delivery_target`` (see ADR 0005).
        """

        async def on_new_message(event: NewMessage.Event) -> None:  # pragma: no cover
            message: Message = event.message  # type: ignore[assignment]
            if message is None:
                return  # Telethon edge case: event has no message (shouldn't happen)
            # Only process genuine channel/group posts and private messages we
            # receive ? outgoing messages (sent by our own account) are ignored.
            if getattr(message, "out", False):
                return

            meta = extract_video_meta(message)
            if meta is None:
                return  # not a video

            channel_id = _peer_id(message.peer_id)
            message_id = message.id
            document_id = _extract_document_id(message)

            # --- watched gate (ADR 0009, Track E) -------------------------------
```

```
            # Opt-in: a source is ingested only when watched=True. An absent
            # sources row means the channel has never been seen ? treat it as
            # unwatched and lazily create a watched=0 row for /channels
            # discoverability (ADR 0009 lines 37-44, 130-131).
            if not await db.should_ingest_source(conn, user.id, channel_id):
                return  # skip unwatched / unknown source

            # --- three-tier dedup (cheapest first) ---

            # Tier 0: same (user, channel, message) instance ? already processed.
            existing = await db.get_video(conn, user.id, channel_id, message_id)
            if existing is not None:
                return

            # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
            if document_id is not None:
                existing = await db.get_video_by_document_id(conn, user.id, document_id)
                if existing is not None:
                    log.info(
                        "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
                        "original_channel=%s original_msg=%s",
                        user.id,
                        document_id,
                        channel_id,
                        message_id,
                        existing.channel_id,
                        existing.message_id,
                    )
                    return

            # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
            if (
                meta.size_bytes is not None
                and meta.duration_s is not None
                and meta.height is not None
                and meta.width is not None
            ):
                existing = await db.get_video_by_content_fingerprint(
                    conn,
                    user.id,
                    meta.size_bytes,
                    meta.duration_s,
                    meta.height,
                    meta.width,
                )
                if existing is not None:
                    log.info(
                        "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
                        "channel=%s msg=%s original_channel=%s original_msg=%s",
                        user.id,
                        meta.size_bytes,
                        meta.duration_s,
                        meta.height,
                        meta.width,
                        channel_id,
                        message_id,
                        existing.channel_id,
                        existing.message_id,
                    )
                    return

            source = await db.get_source_by_provider_id(conn, user.id, "telegram",
str(channel_id))

            # Lazily record the channel (best-effort; failures are non-fatal).
```

```
            try:
                channel = await db.upsert_channel(
                    conn,
                    Channel(user_id=user.id, channel_id=channel_id,
title=_channel_title(event)),
                )
                if channel.id is not None:
                    source = await db.get_source_by_id(conn, channel.id)
            except Exception:
                log.warning(
                    "failed to upsert channel %s for user %s", channel_id, user.id,
exc_info=True
                )

            text = _message_text(message)
            result = engine_holder[ENGINE_KEY].evaluate(meta)
            matched = result.matched
            reasons = list(result.reasons)
            if matched and source is not None:
                topic_result = await _evaluate_topic_preferences(
                    conn,
                    user.id,
                    source,
                    _content_item_preview(
                        user.id,
                        source,
                        message_id,
                        document_id,
                        meta,
                        text,
                        matched,
                    ),
                )
                if topic_result is not None:
                    reasons.append(topic_result.reason)
                    matched = topic_result.passed

            record = VideoRecord(
                user_id=user.id,
                channel_id=channel_id,
                message_id=message_id,
                text=text,
                document_id=document_id,
                duration_s=meta.duration_s,
                width=meta.width,
                height=meta.height,
                size_bytes=meta.size_bytes,
                mime_type=meta.mime_type,
                matched=matched,
            )
            try:
                inserted = await db.insert_video(conn, record)
            except sqlite3.IntegrityError:
                # Lost a dedup race: a concurrent delivery of the same file
                # (same document_id or same message instance) won the insert.
                # The three-tier check above already classified this as new, so
                # the collision means another task is processing (or has
                # processed) it ? log and skip.
                log.info(
                    "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
                    user.id,
                    channel_id,
                    message_id,
                    document_id,
                )
```

```
                return
            log.info(
                "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
                user.id,
                channel_id,
                message_id,
                matched,
                reasons,
            )

            if matched:
                # Build link text for LINK/BOTH modes (None for FORWARD).
                link_text = None
                if delivery_mode != DeliveryMode.FORWARD:
                    channel_title = _channel_title(event)
                    link_text = build_link_text(
                        message_link=getattr(message, "link", None),
                        duration_s=meta.duration_s,
                        height=meta.height,
                        size_bytes=meta.size_bytes,
                        channel_title=channel_title,
                    )
                await deliver(
                    client,
                    message,
                    conn,
                    inserted.id,
                    delivery_mode,
                    link_text=link_text,
                    target=delivery_target,
                )

        return on_new_message


    # --- helpers -----------------------------------------------------------


    def _extract_document_id(message: Message) -> int | None:
        """Extract Telegram's ``document.id`` from a video message.

        Returns None if the message has no document media or the id is absent.
        This is the most reliable content-based dedup key: Telegram reuses the
        same ``document.id`` when a file is forwarded across channels.
        """
        media = getattr(message, "media", None)
        if isinstance(media, MessageMediaDocument):
            media = media.document
        if isinstance(media, Document):
            return getattr(media, "id", None)
        return None


    def _message_text(message: object) -> str | None:
        """Return caption/message text normalized for storage, or None if empty."""
        text = getattr(message, "message", None)
        if text is None:
            text = getattr(message, "raw_text", None)
        if not isinstance(text, str):
            return None
        text = text.strip()
        return text or None


    def _content_item_preview(
```

```
        account_id: int,
        source: Source,
        message_id: int,
        document_id: int | None,
        meta: VideoMeta,
        text: str | None,
        matched: bool,
    ) -> ContentItem:
        """Build the ContentItem shape used by policy stages before insert."""
        assert source.id is not None
        return ContentItem(
            account_id=account_id,
            source_id=source.id,
            provider=source.provider,
            provider_message_id=str(message_id),
            content_type="video",
            text=text,
            document_id=str(document_id) if document_id is not None else None,
            duration_s=meta.duration_s,
            width=meta.width,
            height=meta.height,
            size_bytes=meta.size_bytes,
            mime_type=meta.mime_type,
            dedup_key=db.compute_dedup_key(
                provider=source.provider,
                document_id=str(document_id) if document_id is not None else None,
                source_id=source.id,
                provider_message_id=str(message_id),
                size_bytes=meta.size_bytes,
                duration_s=meta.duration_s,
                height=meta.height,
                width=meta.width,
            ),
            matched=matched,
        )


    async def _evaluate_topic_preferences(
        conn: aiosqlite.Connection,
        account_id: int,
        source: Source,
        content_item: ContentItem,
    ) -> StageResult | None:
        """Evaluate ADR 0012 topic preferences for a candidate item."""
        policy = await db.get_default_policy(conn, account_id)
        if policy is None:
            return None
        return await TopicPreferenceStage().evaluate(
            PolicyContext(content_item=content_item, source=source, policy=policy)
        )


    def _peer_id(peer: object) -> int:
        """Best-effort numeric id for the peer of a message.

        Telegram represents channel/group/user peers as different TL types; we
        collapse them to a single int for dedup/storage. The exact subclass isn't
        important ? only that the id is stable per chat.
        """
        # Each Peer* type has a field like channel_id / chat_id / user_id.
        for field in ("channel_id", "chat_id", "user_id"):
            val = getattr(peer, field, None)
            if val is not None:
                return int(val)
        # Fallback: hash the repr so we never collide with real ids (negative).
```

```
        return -abs(hash(repr(peer)) % (2**31))


    def _channel_title(event: NewMessage.Event) -> str | None:  # pragma: no cover
        """Best-effort channel title from the event, if available."""
        chat = getattr(event, "chat", None)
        return getattr(chat, "title", None) if chat is not None else None


    class _NewMessageCallback:
        """Structural type annotation for the handler returned by make_*_handler."""

        async def __call__(self, event: NewMessage.Event) -> None: ...


    __all__ = [
        "build_client",
        "extract_video_meta",
        "make_new_message_handler",
        "_content_item_preview",
        "_evaluate_topic_preferences",
        "_message_text",
    ]

```

8. assistant (2026-07-06T21:26:10.025Z)

Now let me read the backfill and policy files.

9. user (2026-07-06T21:26:10.703Z)

```
    """Backfill: scan historical channel videos via ``client.iter_messages``.

    Reuses the same pipeline as the live watcher (extract ? dedup ? filter ?
    deliver), so backfill and live mode are seamless ? no duplicates. See
    ``docs/plan.md`` §v1.1 for the feature design.

    Triggered by a ``/backfill [N][hdw]`` command sent to the user's own Saved Messages
    (handled by ``backfill_command_handler``). Calling ``backfill_user`` directly
    from a script is also supported.

    Flood safety:
      - ``iter_messages`` uses ``wait_time=0.5`` between batches.
      - ``FloodWaitError`` from any call is caught and slept on (with jitter).
      - Progress messages to ``me`` are rate-limited (at most one per 30s per dialog).
    """

    from __future__ import annotations

    import asyncio
    import contextlib
    import logging
    import re
    from collections.abc import Awaitable, Callable
    from datetime import UTC, datetime, timedelta

    import aiosqlite
    from telethon import TelegramClient
    from telethon.errors import FloodWaitError

    from tg_video_filter import db
    from tg_video_filter.commands import ENGINE_KEY
    from tg_video_filter.delivery import build_link_text, deliver
    from tg_video_filter.filters import FilterEngine
    from tg_video_filter.models import Channel, DeliveryMode, User, VideoRecord
```

```
    from tg_video_filter.telethon_client import (
        _content_item_preview,
        _evaluate_topic_preferences,
        _extract_document_id,
        _message_text,
        _peer_id,
        extract_video_meta,
    )

    log = logging.getLogger(__name__)

    # Regex to parse /backfill [N][unit] ? e.g. "/backfill", "/backfill 7d", "/backfill 8h",
"/backfill 2w".
    _BACKFILL_RE = re.compile(r"^/backfill(?:\s+(\d+)\s*([hdw]?))?$", re.IGNORECASE)
    _STOP_RE = re.compile(r"^/stop$", re.IGNORECASE)
    _DEFAULT_DAYS = 90
    _MIN_WAIT_BETWEEN_BATCHES = 0.5  # seconds; passed to iter_messages


    def _parse_backfill_days(value_str: str, unit: str) -> float:
        """Parse a backfill value + unit into fractional days.

        >>> _parse_backfill_days("7", "d")
        7.0
        >>> _parse_backfill_days("8", "h")  # doctest: +ELLIPSIS
        0.333...
        >>> _parse_backfill_days("2", "w")
        14.0
        """
        value = int(value_str)
        unit = (unit or "").lower()  # normalize: regex uses re.IGNORECASE
        if value <= 0:
            raise ValueError("backfill duration must be at least 1")
        if unit == "h":
            return value / 24
        elif unit == "w":
            return float(value * 7)
        return float(value)  # days (no unit, or "d")


    def _format_backfill_duration(days: float, *, unit: str = "") -> str:
        """Format a backfill duration for human-readable progress messages.

        When ``unit`` is provided, the original unit is preserved so the user
        sees what they typed (e.g. ``/backfill 25h`` ? ``"25h"``).
        """
        unit = (unit or "").lower()  # normalize, matching _parse_backfill_days contract
        if unit == "h":
            hours = round(days * 24)
            return f"{hours}h"
        if unit == "w":
            weeks = days / 7
            if weeks == int(weeks):
                return f"{int(weeks)}w"
            return f"{weeks:.1f}w"
        # No original unit: format as days.
        if days < 1:
            hours = round(days * 24)
            return f"{hours}h"
        if days == int(days):
            n = int(days)
            return f"{n} day{'s' if n != 1 else ''}"
        return f"{days:.1f} days"


```

```
    async def backfill_user(  # pragma: no cover - requires live Telethon connection
        client: TelegramClient,
        user: User,
        conn: aiosqlite.Connection,
        engine: FilterEngine,
        delivery_mode: DeliveryMode,
        *,
        days: float = _DEFAULT_DAYS,
        delivery_target: str = "me",
        progress_callback: Callable[[str], Awaitable[None]] | None = None,
    ) -> dict[str, int]:
        """Scan the last ``days`` of videos in all of ``user``'s channels.

        Reuses ``extract_video_meta``, the existing ``videos`` dedup table, the
        ``FilterEngine``, and the ``deliver()`` dispatcher. Videos already recorded
        (by the live watcher or a prior backfill) are skipped.

        ``delivery_target`` controls where matches are sent (see ADR 0005);
        progress messages from ``progress_callback`` always go to Saved Messages.

        Progress messages (optional): if ``progress_callback`` is provided, it is
        called with human-readable status strings. The command handler uses this to
        send progress to Saved Messages.

        Returns a summary dict: ``{dialog_count, videos_scanned, videos_matched}``.
        """
        cutoff = datetime.now(UTC) - timedelta(days=days)

        dialogs = await client.get_dialogs()
        summary = {"dialog_count": 0, "videos_scanned": 0, "videos_matched": 0}

        for dialog in dialogs:
            if not getattr(dialog, "is_channel", False) and not getattr(dialog, "is_group",
False):
                continue  # skip personal chats, bots, etc.

            summary["dialog_count"] += 1
            title = getattr(dialog, "title", None) or dialog.name
            log.info("backfill_channel_start channel=%s days=%.2f", title, days)

            if progress_callback:
                await progress_callback(f"? Scanning #{title} (last {days} days)...")

            dialog_videos = 0
            dialog_total = 0
            dialog_matched = 0
            last_progress = asyncio.get_event_loop().time()

            try:
                # NOTE: we do NOT use offset_date because it gets messages OLDER
                # than the given date. Instead we iterate newest-first and manually
                # break when we cross the cutoff boundary.
                async for message in client.iter_messages(
                    dialog.entity,
                    wait_time=_MIN_WAIT_BETWEEN_BATCHES,
                    reverse=False,  # newest-first
                ):
                    # Stop when we reach messages older than the cutoff.
                    msg_date = getattr(message, "date", None)
                    if msg_date is not None and msg_date < cutoff:
                        break

                    dialog_total += 1
                    meta = extract_video_meta(message)  # type: ignore[arg-type]
                    if meta is None:
```

```
                        continue

                    channel_id = _peer_id(getattr(message, "peer_id", None) or 0)
                    message_id = message.id
                    document_id = _extract_document_id(message)  # type: ignore[arg-type]

                    # --- three-tier dedup (cheapest first) ---

                    # Tier 0: same (user, channel, message) instance.
                    existing = await db.get_video(conn, user.id, channel_id, message_id)
                    if existing is not None:
                        continue

                    # Tier 1: same Telegram document.id (forwarded / cross-posted file).
                    if document_id is not None:
                        existing = await db.get_video_by_document_id(conn, user.id,
document_id)
                        if existing is not None:
                            log.info(
                                "backfill_dedup_document_id user=%s doc_id=%s channel=%s
msg=%s "
                                "original_channel=%s original_msg=%s",
                                user.id,
                                document_id,
                                channel_id,
                                message_id,
                                existing.channel_id,
                                existing.message_id,
                            )
                            continue

                    # Tier 2: metadata fingerprint (re-uploaded identical file).
                    if (
                        meta.size_bytes is not None
                        and meta.duration_s is not None
                        and meta.height is not None
                        and meta.width is not None
                    ):
                        existing = await db.get_video_by_content_fingerprint(
                            conn,
                            user.id,
                            meta.size_bytes,
                            meta.duration_s,
                            meta.height,
                            meta.width,
                        )
                        if existing is not None:
                            log.info(
                                "backfill_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s
"
                                "channel=%s msg=%s original_channel=%s original_msg=%s",
                                user.id,
                                meta.size_bytes,
                                meta.duration_s,
                                meta.height,
                                meta.width,
                                channel_id,
                                message_id,
                                existing.channel_id,
                                existing.message_id,
                            )
                            continue

                    summary["videos_scanned"] += 1
                    dialog_videos += 1
```

```
                    source = None
                    try:
                        channel = await db.upsert_channel(
                            conn,
                            Channel(user_id=user.id, channel_id=channel_id, title=title),
                        )
                        if channel.id is not None:
                            source = await db.get_source_by_id(conn, channel.id)
                    except Exception:
                        log.warning(
                            "failed to upsert channel %s for user %s during backfill",
                            channel_id,
                            user.id,
                            exc_info=True,
                        )

                    text = _message_text(message)
                    result = engine.evaluate(meta)
                    matched = result.matched
                    if matched and source is not None:
                        topic_result = await _evaluate_topic_preferences(
                            conn,
                            user.id,
                            source,
                            _content_item_preview(
                                user.id,
                                source,
                                message_id,
                                document_id,  # type: ignore[arg-type]
                                meta,
                                text,
                                matched,
                            ),
                        )
                        if topic_result is not None:
                            matched = topic_result.passed

                    record = VideoRecord(
                        user_id=user.id,
                        channel_id=channel_id,
                        message_id=message_id,
                        text=text,
                        document_id=document_id,  # type: ignore[arg-type]
                        duration_s=meta.duration_s,
                        width=meta.width,
                        height=meta.height,
                        size_bytes=meta.size_bytes,
                        mime_type=meta.mime_type,
                        matched=matched,
                    )
                    inserted = await db.insert_video(conn, record)

                    if matched:
                        summary["videos_matched"] += 1
                        dialog_matched += 1
                        link_text = build_link_text(
                            message_link=getattr(message, "link", None),
                            duration_s=meta.duration_s,
                            height=meta.height,
                            size_bytes=meta.size_bytes,
                            channel_title=title,  # from the dialog object
                        )
                        await deliver(
                            client,
```

```
                            message,
                            conn,
                            inserted.id or 0,
                            delivery_mode,
                            link_text=link_text,
                            target=delivery_target,
                        )

                    # Periodic progress (at most once per 30 seconds per dialog).
                    now = asyncio.get_event_loop().time()
                    if progress_callback and now - last_progress > 30:
                        await progress_callback(
                            f"   #{title}: {dialog_total} msgs, {dialog_videos} videos,
{dialog_matched} matched..."
                        )
                        last_progress = now

            except FloodWaitError as e:
                wait = e.seconds + 1  # small extra buffer
                log.warning("backfill_flood_wait channel=%s seconds=%s", title, e.seconds)
                await asyncio.sleep(wait)

            if progress_callback:
                await progress_callback(
                    f"? #{title}: {dialog_videos} videos, {dialog_matched} matched"
                )

        return summary


    def make_backfill_command_handler(  # pragma: no cover - requires live Telethon
connection
        client: TelegramClient,
        user: User,
        conn: aiosqlite.Connection,
        engine_holder: dict[str, FilterEngine],
        delivery_mode: DeliveryMode,
        delivery_target: str = "me",
    ) -> Callable:
        """Return an async handler for ``/backfill [N][hdw]`` commands in Saved Messages.

        The handler runs ``backfill_user`` in a background task so it doesn't
        block the event loop. Progress messages are sent to ``me``.

        Matches are delivered to ``delivery_target`` (see ADR 0005).

        ``/stop`` cancels the currently running backfill task (if any).

        ``engine_holder`` is a mutable dict so the backfill picks up any
        filter config changes made via ``/set`` commands while it runs.
        """

        _backfill_task: asyncio.Task[None] | None = None

        async def _send_progress(text: str) -> None:
            try:
                await client.send_message("me", text)
            except Exception:
                log.warning("backfill_progress_send_failed", exc_info=True)

        async def on_command(event: object) -> None:
            nonlocal _backfill_task
            message = getattr(event, "message", None)  # type: ignore[union-attr]
            if message is None:
                return
```

```
353
354             # Only respond to messages the user sent to themselves.
355             if not getattr(message, "out", False):
356                 return
357             peer = getattr(message, "peer_id", None)
358             peer_user_id = getattr(peer, "user_id", None) if peer else None
359             if peer_user_id != user.id:
360                 return
361
362             text = getattr(message, "message", "") or ""
363
364             # --- /stop ------------------------------------------------------
365             if _STOP_RE.match(text):
366                 if _backfill_task is not None and not _backfill_task.done():
367                     _backfill_task.cancel()
368                     _backfill_task = None
369                     await _send_progress("? Backfill stopped by user.")
370                 else:
371                     await _send_progress("No backfill is currently running.")
372                 return
373
374             # --- /backfill [N][unit] ----------------------------------------
375             m = _BACKFILL_RE.match(text)
376             if not m:
377                 return
378
379             days_str = m.group(1)
380             unit = (m.group(2) or "").lower()
381             try:
382                 days = _parse_backfill_days(days_str, unit) if days_str else _DEFAULT_DAYS
383             except ValueError as e:
384                 await _send_progress(f"? {e}. Use e.g. `/backfill 8h` or `/backfill 7d`.")
385                 return
386             duration_label = _format_backfill_duration(days, unit=unit)
387
388             await _send_progress(f"? Starting backfill of all channels (last
{duration_label})...")
389
390             # Cancel any previous backfill still running.
391             if _backfill_task is not None and not _backfill_task.done():
392                 _backfill_task.cancel()
393
394             # Run backfill in a background task ? this handler returns immediately.
395             _backfill_task = asyncio.create_task(
396                 _run_backfill_and_report(
397                     client,
398                     user,
399                     conn,
400                     engine_holder,
401                     delivery_mode,
402                     delivery_target,
403                     days,
404                     _send_progress,
405                 ),
406                 name=f"backfill-{user.id}-{duration_label}",
407             )
408
409         return on_command
410
411
412     async def _run_backfill_and_report(  # pragma: no cover - Telethon glue
413         client: TelegramClient,
414         user: User,
415         conn: aiosqlite.Connection,
416         engine_holder: dict[str, FilterEngine],
```

```
        delivery_mode: DeliveryMode,
        delivery_target: str,
        days: float,
        send: Callable,
    ) -> None:
        """Background task: run backfill and report completion via ``send``."""
        try:
            summary = await backfill_user(
                client,
                user,
                conn,
                engine_holder[ENGINE_KEY],
                delivery_mode,
                days=days,
                delivery_target=delivery_target,
                progress_callback=send,
            )
            await send(
                f"? Backfill complete: {summary['dialog_count']} channels scanned, "
                f"{summary['videos_scanned']} videos found, "
                f"{summary['videos_matched']} matched and delivered"
            )
        except asyncio.CancelledError:
            with contextlib.suppress(Exception):
                await send("? Backfill cancelled.")
            raise
        except Exception:
            log.exception("backfill_crashed user=%s days=%d", user.id, days)
            with contextlib.suppress(Exception):
                await send("? Backfill failed due to an error. Check logs for details.")


    __all__ = ["backfill_user", "make_backfill_command_handler"]

```

10. assistant (2026-07-06T21:26:13.391Z)

Now let me read the policy.py TopicPreferenceStage and the db functions involved.

11. user (2026-07-06T21:26:14.369Z)

```
      """Policy pipeline primitives for ADR 0012.

      The policy layer is deliberately small and framework-free in F1: stages receive
      an immutable context, return a structured result, and the evaluator stops on
      the first failure. Later Track F phases add concrete stages.
      """

      from __future__ import annotations

      import json
      from dataclasses import dataclass, field
      from typing import Protocol

      from tg_video_filter.models import ContentItem, Policy, Source


      @dataclass(frozen=True)
      class PolicyContext:
          """Immutable snapshot available to a policy stage."""

          content_item: ContentItem
          source: Source
          policy: Policy
          fetcher: object | None = None
```

```
        budget: object | None = None


    @dataclass(frozen=True)
    class StageResult:
        """Outcome of one policy stage."""

        passed: bool
        reason: str
        stage_name: str
        metadata: dict[str, object] = field(default_factory=dict)

        @staticmethod
        def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
            return StageResult(
                passed=True,
                reason=reason,
                stage_name=stage_name,
                metadata=metadata,
            )

        @staticmethod
        def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
            return StageResult(
                passed=False,
                reason=reason,
                stage_name=stage_name,
                metadata=metadata,
            )


    class PolicyStage(Protocol):
        """A single stateless gate in the policy pipeline."""

        async def evaluate(self, ctx: PolicyContext) -> StageResult:
            """Evaluate ``ctx`` and return a stage result."""
            ...


    @dataclass(frozen=True)
    class TopicPreferences:
        """Parsed ADR 0012 topic preference config."""

        include: tuple[str, ...] = ()
        exclude: tuple[str, ...] = ()
        mode: str = "any"

        @property
        def enabled(self) -> bool:
            return bool(self.include or self.exclude)


    def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
        """Parse ``topics`` from a raw policy config object."""
        raw_topics = config.get("topics")
        if raw_topics is None:
            return TopicPreferences()
        if not isinstance(raw_topics, dict):
            raise ValueError("topics must be a JSON object")
        mode = raw_topics.get("mode", "any")
        if mode not in ("any", "all"):
            raise ValueError("topics.mode must be 'any' or 'all'")
        return TopicPreferences(
            include=_string_tuple(raw_topics.get("include")),
            exclude=_string_tuple(raw_topics.get("exclude")),
```

```
            mode=str(mode),
        )


    def load_topic_preferences(policy: Policy) -> TopicPreferences:
        """Load topic preferences from ``policy.config_json``."""
        decoded = json.loads(policy.config_json)
        if not isinstance(decoded, dict):
            raise ValueError("policy config_json must decode to a JSON object")
        return parse_topic_preferences(decoded)


    def _string_tuple(value: object) -> tuple[str, ...]:
        if value is None:
            return ()
        if not isinstance(value, list):
            raise ValueError("topic include/exclude values must be arrays")
        topics: list[str] = []
        for item in value:
            if not isinstance(item, str):
                raise ValueError("topic include/exclude arrays must contain strings")
            topic = item.strip()
            if topic:
                topics.append(topic)
        return tuple(topics)


    class TopicPreferenceStage:
        """ADR 0012 Tier 2 topic matching over source title and captured caption."""

        stage_name = "topic-preferences"

        async def evaluate(self, ctx: PolicyContext) -> StageResult:
            try:
                prefs = load_topic_preferences(ctx.policy)
            except (json.JSONDecodeError, ValueError) as exc:
                return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
            if not prefs.enabled:
                return StageResult.pass_(self.stage_name, "topic preferences disabled")

            haystack = _topic_haystack(ctx)
            matched_excludes = _matched_topics(prefs.exclude, haystack)
            if matched_excludes:
                return StageResult.fail(
                    self.stage_name,
                    f"excluded topic matched: {matched_excludes[0]}",
                    matched_exclude_topics=list(matched_excludes),
                )

            if not prefs.include:
                return StageResult.pass_(self.stage_name, "no include topics configured")

            matched_includes = _matched_topics(prefs.include, haystack)
            if prefs.mode == "any":
                if matched_includes:
                    return StageResult.pass_(
                        self.stage_name,
                        f"matched include topic: {matched_includes[0]}",
                        matched_include_topics=list(matched_includes),
                    )
                return StageResult.fail(self.stage_name, "no include topics matched")

            missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
            if missing:
```

```
154                 return StageResult.fail(
155                     self.stage_name,
156                     f"missing required topic: {missing[0]}",
157                     matched_include_topics=list(matched_includes),
158                     missing_include_topics=list(missing),
159                 )
160             return StageResult.pass_(
161                 self.stage_name,
162                 "matched all include topics",
163                 matched_include_topics=list(matched_includes),
164             )
165 
166 
167     def _topic_haystack(ctx: PolicyContext) -> str:
168         parts = [ctx.source.title or "", ctx.content_item.text or ""]
169         return "\n".join(parts).casefold()
170 
171 
172     def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
173         return tuple(topic for topic in topics if topic.casefold() in haystack)
174 
175 
176     async def evaluate_pipeline(
177         ctx: PolicyContext,
178         stages: list[PolicyStage],
179     ) -> tuple[bool, list[StageResult]]:
180         """Evaluate stages in order and short-circuit on the first failure."""
181         results: list[StageResult] = []
182         for stage in stages:
183             result = await stage.evaluate(ctx)
184             results.append(result)
185             if not result.passed:
186                 return False, results
187         return True, results
188 
189 
190     __all__ = [
191         "PolicyContext",
192         "PolicyStage",
193         "StageResult",
194         "TopicPreferenceStage",
195         "TopicPreferences",
196         "evaluate_pipeline",
197         "load_topic_preferences",
198         "parse_topic_preferences",
199     ]
200 
```

12. assistant (2026-07-06T21:26:17.095Z)

Now let me look at the diff to focus on exactly what changed, and the relevant db functions.

13. user (2026-07-06T21:26:19.576Z)

```
 src/tg_video_filter/backfill.py        |  52 ++++++++++++--
 src/tg_video_filter/db.py              |   4 ++
 src/tg_video_filter/models.py          |   1 +
 src/tg_video_filter/policy.py          | 117 +++++++++++++++++++++++++++++++++
 src/tg_video_filter/telethon_client.py | 115 ++++++++++++++++++++++++++++++--
 tests/test_db.py                       |   8 +++
 tests/test_policy.py                   | 110 ++++++++++++++++++++++++++++++--
 tests/test_telethon_extraction.py      |  26 +++++--
 8 files changed, 417 insertions(+), 16 deletions(-)
```

14. user (2026-07-06T21:26:24.918Z)

```
diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index 38adc10..e6fca83 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -1573,6 +1573,7 @@ async def get_video_by_content_fingerprint(
         user_id=user_id,
         channel_id=_safe_int_cast(row["src_pid"], "provider_source_id"),
         message_id=_safe_int_cast(row["provider_message_id"], "provider_message_id"),
+        text=row["text"],
         document_id=(
             _safe_int_cast(row["document_id"], "document_id")
             if row["document_id"] is not None
@@ -1677,6 +1678,7 @@ async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) ->
VideoR
         provider="telegram",
         provider_message_id=str(video.message_id),
         content_type="video",
+        text=video.text,
         document_id=str(video.document_id) if video.document_id is not None else None,
         duration_s=video.duration_s,
         width=video.width,
@@ -1730,6 +1732,7 @@ def _content_item_to_video(
         user_id=user_id,
         channel_id=channel_id,
         message_id=_safe_int_cast(item.provider_message_id, "provider_message_id"),
+        text=item.text,
         document_id=(
             _safe_int_cast(item.document_id, "document_id")
             if item.document_id is not None
@@ -1813,6 +1816,7 @@ async def list_videos_for_user(  # v2: used by on-demand /search; not
called in
             user_id=user_id,
             channel_id=_safe_int_cast(r["src_pid"], "provider_source_id"),
             message_id=_safe_int_cast(r["provider_message_id"], "provider_message_id"),
+            text=r["text"],
             document_id=(
                 _safe_int_cast(r["document_id"], "document_id")
                 if r["document_id"] is not None
diff --git a/src/tg_video_filter/models.py b/src/tg_video_filter/models.py
index 4988efe..70e2a8a 100644
--- a/src/tg_video_filter/models.py
+++ b/src/tg_video_filter/models.py
@@ -394,6 +394,7 @@ class VideoRecord(BaseModel):
     user_id: int
     channel_id: int = Field(description="Origin Telegram channel id")
     message_id: int = Field(description="Telegram message id within the channel")
+    text: str | None = Field(default=None, description="Caption / message text")
     document_id: int | None = Field(
         default=None,
         description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
```

15. assistant (2026-07-06T21:26:28.600Z)

Now let me examine key concerns. First, source resolution in `on_new_message` and whether `get_source_by_provider_id` / `get_source_by_id` return a Source with a `title`. Let me look at those db functions and `should_ingest_source`, `get_default_policy`, `get_source_by_id`.

16. user (2026-07-06T21:26:30.464Z)

```
1155:async def get_source_by_provider_id(
1284:async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy | None:
1450:async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1593:async def get_source_by_id(conn: aiosqlite.Connection, source_id: int) -> Source | None:
```

```
1627:async def should_ingest_source(
```

17. user (2026-07-06T21:26:34.792Z)

```
async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
    """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.

    ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
    reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
    New code should call :func:`upsert_source` directly.
    """
    src = await upsert_source(
        conn,
        Source(
            account_id=channel.user_id,
            provider="telegram",
            provider_source_id=str(channel.channel_id),
            title=channel.title,
            kind="channel",
            watched=False,
            seen_first_at=channel.seen_first_at,
        ),
    )
    return _source_to_channel(src)


async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) -> Channel |
None:
    """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
    src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
    return _source_to_channel(src) if src else None


def _source_to_channel(src: Source) -> Channel:
    """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`.

    Only valid for Telegram sources (``provider == \"telegram\"``); the legacy
    ``Channel.channel_id`` is a Telegram-specific int id. Non-Telegram sources
    raise ``ValueError`` ? new code should use :class:`Source` directly instead
    of the deprecated :class:`Channel` shim (ADR 0011 provider-neutrality guard).
    """
    if src.provider != "telegram":
        raise ValueError(
            f"Cannot project {src.provider!r} source onto the Telegram-specific "
            f"Channel type. Use Source directly (ADR 0011)."
        )
    return Channel(
        id=src.id,
        user_id=src.account_id,
        channel_id=int(src.provider_source_id),
        title=src.title,
        seen_first_at=src.seen_first_at,
    )
```

--- legacy VideoRecord shims (backwards-compatible view over content_items) -

```
async def get_video(
    conn: aiosqlite.Connection, user_id: int, channel_id: int, message_id: int
) -> VideoRecord | None:
    """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).

    Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
```

```
    :class:`Source` + :class:`ContentItem` and projects the latter onto a
    :class:`VideoRecord`. New code should call :func:`get_content_item_by_message`.
    """
    src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
    if src is None:
        return None
    item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))
    return _content_item_to_video(item, src, user_id, channel_id) if item else None


async def get_video_by_document_id(
    conn: aiosqlite.Connection, user_id: int, document_id: int
) -> VideoRecord | None:
    """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
    dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
    item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
    if item is None:
        return None
    src = await get_source_by_id(conn, item.source_id)
    if src is None:
        return None
    return _content_item_to_video(item, src, user_id, int(src.provider_source_id))


async def get_video_by_content_fingerprint(
    conn: aiosqlite.Connection,
    user_id: int,
    size_bytes: int,
    duration_s: int,
    height: int,
    width: int,
) -> VideoRecord | None:
    """Tier-2 content dedup: query metadata COLUMNS, not the ``dedup_key``.

    ADR 0011: ``dedup_key`` is a storage discriminator (``tg:doc:X`` vs
    ``tg:fp:...`` vs ``tg:msg:...``), NOT the read path. Items with a
    ``document_id`` are stored under ``tg:doc:X`` but their metadata columns
    are still populated ? querying the columns catches cross-key matches
    that a key-based lookup would miss (the Tier-2 regression: a re-upload
    with a new doc id + matching fingerprint must still be caught).

    .. versionchanged:: Track B (review fix)
       Reverted from key-based lookup to column-based lookup, matching the v1
       pre-ADR-0011 behaviour that was regressed when :func:`compute_dedup_key`
       made the three tiers mutually-exclusive keys.
    """
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        """SELECT ci.*, s.provider_source_id AS src_pid
           FROM content_items ci
           JOIN sources s ON s.id = ci.source_id
           WHERE ci.account_id = ?
             AND ci.size_bytes = ?
             AND ci.duration_s = ?
             AND ci.height = ?
             AND ci.width = ?
           LIMIT 1""",
        (user_id, size_bytes, duration_s, height, width),
    ) as cur:
        row = await cur.fetchone()
    if row is None:
        return None
    return VideoRecord(
        id=row["id"],
        user_id=user_id,
```

```
        channel_id=_safe_int_cast(row["src_pid"], "provider_source_id"),
        message_id=_safe_int_cast(row["provider_message_id"], "provider_message_id"),
        text=row["text"],
        document_id=(
            _safe_int_cast(row["document_id"], "document_id")
            if row["document_id"] is not None
            else None
        ),
        duration_s=row["duration_s"],
        width=row["width"],
        height=row["height"],
        size_bytes=row["size_bytes"],
        mime_type=row["mime_type"],
        matched=bool(row["matched"]),
        forwarded=False,
        seen_at=row["seen_at"],
    )


async def get_source_by_id(conn: aiosqlite.Connection, source_id: int) -> Source | None:
    """Resolve a source by its surrogate PK (used by the legacy video shims)."""
    conn.row_factory = aiosqlite.Row
    async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
        row = await cur.fetchone()
    return _row_to_source(row) if row else None


async def set_source_watched(
    conn: aiosqlite.Connection, account_id: int, source_id: int, watched: bool
) -> None:
    """Toggle the ``watched`` flag on a source (ADR 0009, Track E)."""
    await conn.execute(
        "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
        (int(watched), source_id, account_id),
    )
    await conn.commit()


async def list_account_sources(
    conn: aiosqlite.Connection, account_id: int, *, watched_only: bool = False
) -> list[Source]:
    """List all sources for an account, optionally filtering to watched only."""
    conn.row_factory = aiosqlite.Row
    query = "SELECT * FROM sources WHERE account_id = ?"
    params: tuple = (account_id,)
    if watched_only:
        query += " AND watched = 1"
    query += " ORDER BY watched DESC, provider_source_id ASC"
    async with conn.execute(query, params) as cur:
        rows = await cur.fetchall()
    return [_row_to_source(r) for r in rows]


async def should_ingest_source(
    conn: aiosqlite.Connection, account_id: int, channel_id: int
) -> bool:
    """Return True if a channel should be ingested (ADR 0009 opt-in gate).

    - Absent sources row: create it lazily with ``watched=False`` for
      ``/channels`` discoverability; return False (not yet watched).
    - Existing row: respect the ``watched`` flag.

    Pure helper extracted from ``telethon_client.py`` for testability.
    """
    src = await get_source_by_provider_id(conn, account_id, "telegram", str(channel_id))
```

```
    if src is None:
        # First-ever message from this channel ? create a discoverable
        # unwatched row so /channels can list it (ADR 0009 lines 48-54).
        await upsert_source(
            conn,
            Source(
                account_id=account_id,
                provider="telegram",
                provider_source_id=str(channel_id),
                kind="channel",
                watched=False,
            ),
        )
        return False
    return src.watched


async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
    """Deprecated shim: insert into ``content_items`` (ADR 0011).

    Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
    if absent), computes the provider-neutral ``dedup_key`` via
```

18. assistant (2026-07-06T21:26:37.402Z)

Now I have a critical observation. Let me look at `upsert_source` to check the watched-flag clobbering concern in backfill.

19. user (2026-07-06T21:26:38.584Z)

```
1108:async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
```

20. user (2026-07-06T21:26:41.141Z)

```
1108    async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
1109        """Insert a source row, or refresh title/last_seen_at on conflict.
1110
1111        The conflict key is ``(account_id, provider, provider_source_id)`` per the
1112        ADR 0011 schema. Returns the persisted row.
1113
1114        ``watched`` is **user-controlled state** (toggled only via
1115        :func:`set_source_watched` / the ``/watch`` and ``/unwatch`` commands) and is
1116        therefore **never overwritten on conflict** ? the ingestion path
1117        (``upsert_channel`` / ``insert_video``) re-touches a source on every message,
1118        so clobbering ``watched`` here would silently un-watch a channel the user
1119        opted into after its first ingested video (ADR 0009). ``watched`` is only set
1120        on the initial INSERT of a new row. ``title`` is refreshed but never nulled:
1121        the ingestion shims pass ``title=None``, so ``COALESCE`` keeps the existing
1122        title rather than erasing the channel name.
1123        """
1124        await conn.execute(
1125            """INSERT INTO sources
1126                 (account_id, provider, provider_source_id, title, kind, watched,
1127                  seen_first_at, last_seen_at)
1128               VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1129               ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1130                 title=COALESCE(excluded.title, sources.title),
1131                 last_seen_at=excluded.last_seen_at""",
1132            (
1133                source.account_id,
1134                source.provider,
1135                source.provider_source_id,
1136                source.title,
1137                source.kind,
```

```
                int(source.watched),
                source.seen_first_at.isoformat(),
                (source.last_seen_at.isoformat() if source.last_seen_at else None),
            ),
        )
        await conn.commit()
        conn.row_factory = aiosqlite.Row
        async with conn.execute(
            """SELECT * FROM sources
               WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
            (source.account_id, source.provider, source.provider_source_id),
        ) as cur:
            row = await cur.fetchone()
        assert row is not None  # just upserted by the conflict key
        return _row_to_source(row)


    async def get_source_by_provider_id(
        conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
    ) -> Source | None:
        """Resolve a source by its provider-native id."""
        conn.row_factory = aiosqlite.Row
        async with conn.execute(
            """SELECT * FROM sources
               WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
            (account_id, provider, provider_source_id),
        ) as cur:
            row = await cur.fetchone()
        return _row_to_source(row) if row else None


    # --- ContentItem repos (ADR 0011) ------------------------------------------


    async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) ->
ContentItem:
        """Insert a content item. Raises ``IntegrityError`` on a dedup collision.

        The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
        unique constraints guard against duplicates:
          - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
          - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
        """
        cur = await conn.execute(
            """INSERT INTO content_items
                 (account_id, source_id, provider, provider_message_id, content_type,
                  text, link_url, media_ref, document_id, duration_s, width, height,
                  size_bytes, mime_type, dedup_key, matched, seen_at)
               VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
            (
                item.account_id,
                item.source_id,
                item.provider,
                item.provider_message_id,
                item.content_type,
                item.text,
                item.link_url,
                item.media_ref,
                item.document_id,
                item.duration_s,
                item.width,
                item.height,
                item.size_bytes,
                item.mime_type,
                item.dedup_key,
```

```
                int(item.matched),
                item.seen_at.isoformat(),
            ),
        )
        await conn.commit()
        return item.model_copy(update={"id": cur.lastrowid})


    async def get_content_item_by_message(
        conn: aiosqlite.Connection,
        account_id: int,
        source_id: int,
        provider_message_id: str,
    ) -> ContentItem | None:
        """Tier-0 lookup by the (account, source, provider-message-id) natural key."""
        conn.row_factory = aiosqlite.Row
        async with conn.execute(
            """SELECT * FROM content_items
               WHERE account_id = ? AND source_id = ? AND provider_message_id = ?""",
            (account_id, source_id, provider_message_id),
        ) as cur:
            row = await cur.fetchone()
        return _row_to_content_item(row) if row else None


    async def get_content_item_by_dedup_key(
        conn: aiosqlite.Connection, account_id: int, dedup_key: str
    ) -> ContentItem | None:
        """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).

        Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
        a single query: the ``dedup_key`` already encodes whichever tier was
        applicable at ingestion time.
        """
        conn.row_factory = aiosqlite.Row
        async with conn.execute(
            "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
            (account_id, dedup_key),
        ) as cur:
            row = await cur.fetchone()
        return _row_to_content_item(row) if row else None


    # --- Policy repos (ADR 0011, Option 2A) ------------------------------------


    async def upsert_policy(conn: aiosqlite.Connection, policy: Policy) -> Policy:
        """Insert or update a policy row by ``(account_id, name)``.

        Exactly one policy per account should carry ``is_default=True``; callers
        are responsible for flipping the previous default before flipping a new
        one (enforced by a partial unique index landed by future work, not in the
        schema ? SQLite ``CHECK`` can't express "at most one per group").
        """
        await conn.execute(
            """INSERT INTO policies (account_id, name, config_json, is_default,
                                     created_at, updated_at)
               VALUES (?, ?, ?, ?, ?, ?)
               ON CONFLICT(account_id, name) DO UPDATE SET
                 config_json=excluded.config_json,
                 is_default=excluded.is_default,
                 updated_at=excluded.updated_at""",
            (
                policy.account_id,
                policy.name,
```

```
1267                policy.config_json,
1268                int(policy.is_default),
1269                policy.created_at.isoformat(),
1270                policy.updated_at.isoformat(),
1271            ),
1272        )
1273        await conn.commit()
1274        conn.row_factory = aiosqlite.Row
1275        async with conn.execute(
1276            "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1277            (policy.account_id, policy.name),
1278        ) as cur:
1279            row = await cur.fetchone()
1280        assert row is not None
1281        return _row_to_policy(row)
1282
1283
1284    async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy |
None:
1285        """Return the account's default policy (``is_default=1``)."""
1286        conn.row_factory = aiosqlite.Row
1287        async with conn.execute(
1288            "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1289            (account_id,),
1290        ) as cur:
1291            row = await cur.fetchone()
1292        return _row_to_policy(row) if row else None
1293
1294
1295    def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1296        """Decode a policy config JSON object.
1297
1298        ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
1299        expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
1300        nested stage config can coexist.
1301        """
1302        decoded = json.loads(raw)
1303        if not isinstance(decoded, dict):
1304            raise ValueError("policy config_json must decode to a JSON object")
1305        return decoded
1306
1307
```

21. assistant (2026-07-06T21:26:49.864Z)

The `upsert_source` watched-flag safety is confirmed - it never overwrites watched on conflict. So backfill's `upsert_channel` (which sets watched=False in the Source) is safe on existing rows. Good.

Now the critical concern: **the `matched` field on the throwaway ContentItem and the dedup_key**. Note that in `_content_item_preview`, `compute_dedup_key` is called. But more importantly, let me examine a subtle divergence. In `on_new_message`, `source` is resolved first via `get_source_by_provider_id` (line 255), then re-resolved after upsert. Let me look at `_row_to_source` to confirm `title` is populated on the Source used in the topic haystack.

Now, a key question ? **does the topic evaluation use the source's `title`?** The source returned by `get_source_by_id` after `upsert_channel`. On an existing watched channel, the title is preserved (COALESCE). Good.

Let me now focus on a real potential bug: **the reason `None` append**. Let me check whether `topic_result.reason` can ever be None. Looking at `StageResult`, `reason` is always a string. So no None issue there.

Let me now run a repro to test the actual wiring semantics ? particularly the include-mode "all" with an empty haystack, and the exclude veto. But first, the most important divergence: let me

check the `mode="all"` logic against a substring match nuance.

22. user (2026-07-06T21:26:51.794Z)

78c4f8f feat(policy): add F2 topic preferences

23. assistant (2026-07-06T21:26:58.759Z)

Now let me examine the "all" mode logic carefully in policy.py (lines 152-159). `missing = tuple(topic for topic in prefs.include if topic not in matched_includes)`. Here `matched_includes` is computed by `_matched_topics` which returns topics whose `.casefold()` is in the haystack ? but returns the **original** topic string, not casefolded. And `topic not in matched_includes` compares the original include topic against the original matched topics. That's consistent (same source list). Fine.

Now let me examine a real concern about the AND semantics and whether topic-rejected items can still be delivered. Let me trace: in both handlers, `matched = topic_result.passed` only runs when `topic_result is not None`. If `get_default_policy` returns None (no default policy), `_evaluate_topic_preferences` returns None, so `matched` stays as metadata-match result ? item delivered on metadata alone. That's intended (topics disabled when no policy).

Now the key wiring divergence question. Let me write a repro that actually exercises the topic stage through the DB, to confirm behavior and check the `matched` on the throwaway item.

24. user (2026-07-06T21:26:59.816Z)

```
1660        if absent), computes the provider-neutral ``dedup_key`` via
1661        :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1662        ``VideoRecord`` shape is reconstructed from the inserted row.
1663        """
1664        src = await upsert_source(
1665            conn,
1666            Source(
1667                account_id=video.user_id,
1668                provider="telegram",
1669                provider_source_id=str(video.channel_id),
1670                kind="channel",
1671                watched=False,
1672            ),
1673        )
1674        assert src.id is not None  # upsert_source populates the surrogate PK
1675        item = ContentItem(
1676            account_id=video.user_id,
1677            source_id=src.id,
1678            provider="telegram",
1679            provider_message_id=str(video.message_id),
1680            content_type="video",
1681            text=video.text,
1682            document_id=str(video.document_id) if video.document_id is not None else None,
1683            duration_s=video.duration_s,
1684            width=video.width,
1685            height=video.height,
1686            size_bytes=video.size_bytes,
1687            mime_type=video.mime_type,
1688            dedup_key=compute_dedup_key(
1689                provider="telegram",
1690                document_id=(str(video.document_id) if video.document_id is not None else
None),
1691                source_id=src.id,
1692                provider_message_id=str(video.message_id),
1693                size_bytes=video.size_bytes,
1694                duration_s=video.duration_s,
1695                height=video.height,
1696                width=video.width,
```

```
1697            ),
1698            matched=video.matched,
1699            seen_at=video.seen_at,
1700        )
1701        item = await insert_content_item(conn, item)
1702        return _content_item_to_video(item, src, video.user_id, video.channel_id)
1703
1704
1705    async def mark_forwarded(conn: aiosqlite.Connection, video_id: int) -> None:
1706        """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1707
1708        Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1709        #28). For Phase 1 (before Track C lands) there is no equivalent column on
1710        ``content_items`` ? the ``matched`` flag alone is sufficient because v1
1711        delivery still targets the single ``delivery_target`` and retried delivery
1712        is harmless (forwards are idempotent). This shim is retained so existing
1713        call sites in ``delivery.py`` keep working without a flag day.
1714        """
1715        # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
1716        # The row's ``matched`` flag (set at insert time) is the surviving signal.
1717        return None
1718
1719
1720    def _content_item_to_video(
1721        item: ContentItem, src: Source, user_id: int, channel_id: int
1722    ) -> VideoRecord:
1723        """Project a :class:`ContentItem` onto the legacy :class:`VideoRecord`.
1724
1725        ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;
1726        per-destination delivery state moves to Track C). Existing callers read it
1727        only to decide whether to skip re-delivery; since forwards are idempotent
1728        the ``False`` default is safe for Phase 1.
1729        """
1730        return VideoRecord(
1731            id=item.id,
1732            user_id=user_id,
1733            channel_id=channel_id,
1734            message_id=_safe_int_cast(item.provider_message_id, "provider_message_id"),
1735            text=item.text,
1736            document_id=(
1737                _safe_int_cast(item.document_id, "document_id")
1738                if item.document_id is not None
1739                else None
```

25. assistant (2026-07-06T21:27:04.011Z)

Now I understand the full flow. Let me write a repro that exercises the whole live-path wiring semantics through the db + policy, to confirm the topic gating behaves as claimed and to probe edge cases. Let me check the schema/setup helpers first.

26. user (2026-07-06T21:27:06.136Z)

```
200:        await _apply_schema(conn)
203:async def _apply_schema(conn: aiosqlite.Connection) -> None:
522:    # dropped by _apply_schema post-003). Nothing to backfill.
```

27. assistant (2026-07-06T21:27:22.651Z)

```
import asyncio
import json
import aiosqlite
from datetime import UTC, datetime

from tg_video_filter import db
```

```
from tg_video_filter.models import Source, Policy, ContentItem
from tg_video_filter.policy import TopicPreferenceStage, PolicyContext
from tg_video_filter.telethon_client import _evaluate_topic_preferences, _content_item_preview
from tg_video_filter.filters import VideoMeta


async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)

    acct = 111
    # Create a watched source with a title.
    src = await db.upsert_source(conn, Source(
        account_id=acct, provider="telegram", provider_source_id="5000",
        title="Cooking Channel", kind="channel", watched=True,
    ))
    print("source id", src.id, "title", src.title, "watched", src.watched)

    # Default policy with include topics.
    now = datetime.now(UTC)
    await db.upsert_policy(conn, Policy(
        account_id=acct, name="default",
        config_json=json.dumps({"topics": {"include": ["recipe"], "mode": "any"}}),
        is_default=True, created_at=now, updated_at=now,
    ))

    meta = VideoMeta(size_bytes=100, duration_s=10, height=200, width=300,
mime_type="video/mp4")

    # Case A: caption contains 'recipe' -> should pass
    item = _content_item_preview(acct, src, 1, 999, meta, "Best recipe ever", True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("A pass?", r.passed, r.reason)

    # Case B: caption None, title has no topic -> should fail (include not matched)
    item = _content_item_preview(acct, src, 2, 998, meta, None, True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("B pass?", r.passed, r.reason)

    # Case C: title itself contains topic
    src2 = await db.upsert_source(conn, Source(
        account_id=acct, provider="telegram", provider_source_id="6000",
        title="Recipe Hub", kind="channel", watched=True,
    ))
    item = _content_item_preview(acct, src2, 3, 997, meta, None, True)
    r = await _evaluate_topic_preferences(conn, acct, src2, item)
    print("C pass?", r.passed, r.reason)

    # Case D: no default policy -> None (topics skipped)
    conn2 = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn2)
    src3 = await db.upsert_source(conn2, Source(
        account_id=acct, provider="telegram", provider_source_id="7000",
        title="X", kind="channel", watched=True,
    ))
    item = _content_item_preview(acct, src3, 4, 996, meta, "recipe", True)
    r = await _evaluate_topic_preferences(conn2, acct, src3, item)
    print("D result (None means skip):", r)

    # Case E: exclude veto
    await db.upsert_policy(conn, Policy(
        account_id=acct, name="default",
        config_json=json.dumps({"topics": {"exclude": ["spam"]}}),
        is_default=True, created_at=now, updated_at=now,
    ))
```

```
    item = _content_item_preview(acct, src, 5, 995, meta, "recipe spam", True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("E pass? (should be False - veto)", r.passed, r.reason)

    await conn.close()
    await conn2.close()


asyncio.run(main())
```

28. user (2026-07-06T21:27:22.651Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

29. user (2026-07-06T21:27:27.515Z)

```
removed
```

30. assistant (2026-07-06T21:27:40.013Z)

```
import asyncio
import json
import aiosqlite
from datetime import UTC, datetime

from tg_video_filter import db
from tg_video_filter.models import Source, Policy
from tg_video_filter.telethon_client import _evaluate_topic_preferences, _content_item_preview
from tg_video_filter.filters import VideoMeta


async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)

    acct = 111
    src = await db.upsert_source(conn, Source(
        account_id=acct, provider="telegram", provider_source_id="5000",
        title="Cooking Channel", kind="channel", watched=True,
    ))
    print("source id", src.id, "title", src.title, "watched", src.watched)

    now = datetime.now(UTC)
    await db.upsert_policy(conn, Policy(
        account_id=acct, name="default",
        config_json=json.dumps({"topics": {"include": ["recipe"], "mode": "any"}}),
        is_default=True, created_at=now, updated_at=now,
    ))

    meta = VideoMeta(size_bytes=100, duration_s=10, height=200, width=300,
mime_type="video/mp4")

    item = _content_item_preview(acct, src, 1, 999, meta, "Best recipe ever", True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("A pass?", r.passed, r.reason)

    item = _content_item_preview(acct, src, 2, 998, meta, None, True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("B pass?", r.passed, r.reason)

    src2 = await db.upsert_source(conn, Source(
        account_id=acct, provider="telegram", provider_source_id="6000",
        title="Recipe Hub", kind="channel", watched=True,
    ))
```

```
    item = _content_item_preview(acct, src2, 3, 997, meta, None, True)
    r = await _evaluate_topic_preferences(conn, acct, src2, item)
    print("C pass?", r.passed, r.reason)

    conn2 = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn2)
    src3 = await db.upsert_source(conn2, Source(
        account_id=acct, provider="telegram", provider_source_id="7000",
        title="X", kind="channel", watched=True,
    ))
    item = _content_item_preview(acct, src3, 4, 996, meta, "recipe", True)
    r = await _evaluate_topic_preferences(conn2, acct, src3, item)
    print("D result (None means skip):", r)

    await db.upsert_policy(conn, Policy(
        account_id=acct, name="default",
        config_json=json.dumps({"topics": {"exclude": ["spam"]}}),
        is_default=True, created_at=now, updated_at=now,
    ))
    item = _content_item_preview(acct, src, 5, 995, meta, "recipe spam", True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("E pass? (should be False - veto)", r.passed, r.reason)

    await conn.close()
    await conn2.close()


asyncio.run(main())
```

31. user (2026-07-06T21:27:40.042Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro.py (file state
is current in your context ? no need to Read it back)
```

32. user (2026-07-06T21:29:44.357Z)

```
Command running in background with ID: bfij326hq. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bfij326hq.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

33. user (2026-07-06T21:29:52.015Z)

```
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 71, in <module>
    asyncio.run(main())
    ~~~~~~~~~~~^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
           ~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
           ~~~~~~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 17, in main
    src = await db.upsert_source(conn, Source(
```

```
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    ...<2 lines>...
    ))
    ^^
  File "C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py", line 1124, in upsert_source
    await conn.execute(
    ...<17 lines>...
    )
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223,
in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 160,
in _execute
    return await future
           ^^^^^^^^^^^^
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in
_connection_worker_thread
    result = function()
sqlite3.IntegrityError: FOREIGN KEY constraint failed
```

34. assistant (2026-07-06T21:29:54.264Z)

Needs an accounts row. Let me check how tests set up an account.

35. user (2026-07-06T21:29:56.509Z)

```
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\bootstrap.py:65:        await
db.upsert_account(conn, Account(id=user.id, telegram_id=user.id))
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:397:        INSERT INTO accounts (id,
telegram_id, provider, created_at)
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:940:async def upsert_account(conn:
aiosqlite.Connection, account: Account) -> Account:
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:943:        """INSERT INTO accounts
(id, telegram_id, provider, display_name, created_at)
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:961:async def get_account(conn:
aiosqlite.Connection, account_id: int) -> Account | None:
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1071:    await upsert_account(conn,
account)
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1843:    "upsert_account",
C:\Users\avidu\Projects\_wt-pr51\tests\test_db.py:109:async def _seed_account(conn:
aiosqlite.Connection, aid: int = USER_ID) -> Account:
C:\Users\avidu\Projects\_wt-pr51\tests\test_db.py:110:    return await db.upsert_account(conn,
Account(id=aid, telegram_id=aid))
C:\Users\avidu\Projects\_wt-pr51\tests\test_db.py:113:async def
test_upsert_and_get_account(conn: aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\_wt-pr51\tests\test_db.py:114:    created = await db.upsert_account(
C:\Users\avidu\Projects\_wt-pr51\tests\test_db.py:126:async def
test_get_account_returns_none_when_absent(conn: aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\_wt-pr51\tests\test_db.py:134:    await db.upsert_account(conn,
Account(id=USER_ID, telegram_id=USER_ID))
C:\Users\avidu\Projects\_wt-pr51\tests\test_db.py:143:    await _seed_account(conn)
C:\Users\avidu\Projects\_wt-pr51\tests\test_db.py:167:    await db.upsert_account(conn,
Account(id=1, telegram_id=1))

[Showing results with pagination = limit: 15]
```

36. user (2026-07-06T21:30:05.604Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

37. user (2026-07-06T21:30:09.636Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro.py has been updated successfully. (file state is current in your context ? no need to Read it back)

38. user (2026-07-06T21:30:16.733Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro.py has been updated successfully. (file state is current in your context ? no need to Read it back)

39. user (2026-07-06T21:30:23.278Z)

```
source id 1 title Cooking Channel watched True
A pass? True matched include topic: recipe
B pass? False no include topics matched
C pass? True matched include topic: recipe
D result (None means skip): None
E pass? (should be False - veto) False excluded topic matched: spam
```

40. assistant (2026-07-06T21:30:41.276Z)

Core semantics confirmed: exclude veto works, include matching works, title contributes to haystack, no-policy skips. These match the reviewer's claims ? no defect there.

Now let me hunt for the actual **wiring divergence between live and backfill**. Comparing the two carefully:

Live (telethon_client.py:274-291):
```python
if topic_result is not None:
    reasons.append(topic_result.reason)
    matched = topic_result.passed
```

Backfill (backfill.py:260-261):
```python
if topic_result is not None:
    matched = topic_result.passed
```

Both gate delivery on final `matched`. Both turn matched True?False only. This is consistent.

Now let me examine a **real bug**: the source resolution in `on_new_message`. Note lines 255-268. `source` is first fetched at line 255. Then upsert_channel runs and re-fetches. But there's an ordering issue ? let me check the `should_ingest_source` gate interaction. `should_ingest_source` at line 198 creates a watched=False row if absent, but returns False (skip) in that case. So if we pass the gate, the source row already exists and is watched. So `source` at line 255 will be non-None for any item that passes the gate. Good ? topics always run for watched channels. Let me verify there's no path where source is None after passing the gate.

Actually there IS a subtle concern. Let me re-examine: at line 255 `get_source_by_provider_id` returns the source. Then the try-block re-fetches via upsert. If upsert_channel throws, `source` retains the line-255 value (non-None). So source is reliably non-None. Good.

Now, the more interesting question: **does the throwaway ContentItem's `dedup_key` / `matched` matter?** It's only used for topic eval and discarded. The haystack uses only `source.title` and `content_item.text`. The `matched`, `dedup_key`, etc. are irrelevant to TopicPreferenceStage. So no effect. Confirmed harmless.

Now let me look harder at a potential real issue ? the **reasons logging in backfill vs live**. Live appends `topic_result.reason`; backfill does NOT append it. That's a minor logging divergence but not a delivery bug. Let me note it as low.

Now, the most important thing: let me check whether there's an issue with `_message_text` and `raw_text` on the **backfill** path where messages come from `iter_messages` (real Message objects). Let me check the config.py / policy default ? is there actually a default policy created for users? If F1 doesn't create a default policy by default, topics never run. But that's F1 scope.

Let me look for a genuinely impactful bug: the `matched` passed to insert. Let me re-examine ? in backfill, `summary["videos_matched"]` and delivery both use final `matched`. Consistent.

Let me check one more thing ? whether `text` (caption) is used in the dedup fingerprint or affects dedup. It's not. Good.

Now let me verify the exclude-veto interaction with metadata match more carefully via the actual handler logic, and probe the **empty-string title** and **casefold** edge. Let me also check: what if `include` topics exist but `text` is None and title is None ? mode "all"?

41. user (2026-07-06T21:30:43.557Z)

```
diff --git a/tests/test_policy.py b/tests/test_policy.py
index 02234a7..18e64cf 100644
--- a/tests/test_policy.py
+++ b/tests/test_policy.py
@@ -3,15 +3,28 @@
 from __future__ import annotations

 from tg_video_filter.models import ContentItem, Policy, Source
-from tg_video_filter.policy import PolicyContext, StageResult, evaluate_pipeline
-
-
-def _ctx() -> PolicyContext:
+from tg_video_filter.policy import (
+    PolicyContext,
+    StageResult,
+    TopicPreferences,
+    TopicPreferenceStage,
+    evaluate_pipeline,
+    parse_topic_preferences,
+)
+
+
+def _ctx(
+    *,
+    policy_json: str = "{}",
+    source_title: str | None = None,
+    text: str | None = None,
+) -> PolicyContext:
     return PolicyContext(
         content_item=ContentItem(
             account_id=111,
             source_id=1,
             provider_message_id="42",
+            text=text,
             dedup_key="tg:msg:1:42",
         ),
         source=Source(
@@ -19,12 +32,13 @@ def _ctx() -> PolicyContext:
             account_id=111,
             provider="telegram",
             provider_source_id="-1001",
+            title=source_title,
         ),
         policy=Policy(
             id=1,
             account_id=111,
             name="default",
```

```
-            config_json="{}",
+            config_json=policy_json,
             is_default=True,
         ),
     )
@@ -99,3 +113,89 @@ def test_stage_result_helpers_attach_metadata() -> None:
     assert passed.metadata == {"duration_s": 60}
     assert failed.passed is False
     assert failed.metadata == {"topic": "crypto"}
+
+
+# --- TopicPreferenceStage ----------------------------------------------------
+
+
+def test_parse_topic_preferences_defaults_to_disabled() -> None:
+    assert parse_topic_preferences({}) == TopicPreferences()
+
+
+def test_parse_topic_preferences_rejects_invalid_mode() -> None:
+    try:
+        parse_topic_preferences({"topics": {"mode": "sometimes"}})
+    except ValueError as exc:
+        assert "topics.mode" in str(exc)
+    else:  # pragma: no cover - assertion guard
+        raise AssertionError("invalid topic mode should raise")
+
+
+async def test_topic_preferences_disabled_passes() -> None:
+    result = await TopicPreferenceStage().evaluate(_ctx(text="anything"))
+
+    assert result.passed is True
+    assert result.reason == "topic preferences disabled"
+
+
+async def test_topic_preferences_include_any_matches_source_title() -> None:
+    ctx = _ctx(
+        policy_json='{"topics":{"include":["AI","Philosophy"],"mode":"any"}}',
+        source_title="Daily AI Clips",
+    )
+
+    result = await TopicPreferenceStage().evaluate(ctx)
+
+    assert result.passed is True
+    assert result.metadata == {"matched_include_topics": ["AI"]}
+
+
+async def test_topic_preferences_include_all_matches_caption_and_source() -> None:
+    ctx = _ctx(
+        policy_json='{"topics":{"include":["AI","Philosophy"],"mode":"all"}}',
+        source_title="AI Clips",
+        text="A philosophy lecture",
+    )
+
+    result = await TopicPreferenceStage().evaluate(ctx)
+
+    assert result.passed is True
+    assert result.reason == "matched all include topics"
+    assert result.metadata == {"matched_include_topics": ["AI", "Philosophy"]}
+
+
+async def test_topic_preferences_include_all_fails_when_missing_topic() -> None:
+    ctx = _ctx(
+        policy_json='{"topics":{"include":["AI","Philosophy"],"mode":"all"}}',
+        text="AI news",
+    )
```

```
+
+    result = await TopicPreferenceStage().evaluate(ctx)
+
+    assert result.passed is False
+    assert result.metadata == {
+        "matched_include_topics": ["AI"],
+        "missing_include_topics": ["Philosophy"],
+    }
+
+
+async def test_topic_preferences_exclude_vetoes_include_match() -> None:
+    ctx = _ctx(
+        policy_json='{"topics":{"include":["AI"],"exclude":["Crypto"],"mode":"any"}}',
+        text="AI crypto market explainer",
+    )
+
+    result = await TopicPreferenceStage().evaluate(ctx)
+
+    assert result.passed is False
+    assert result.reason == "excluded topic matched: Crypto"
+    assert result.metadata == {"matched_exclude_topics": ["Crypto"]}
+
+
+async def test_topic_preferences_invalid_config_fails_explainably() -> None:
+    result = await TopicPreferenceStage().evaluate(
+        _ctx(policy_json='{"topics":{"include":"AI"}}', text="AI")
+    )
+
```

42. assistant (2026-07-06T21:30:50.667Z)

Now here's a subtle but important finding. Look at `test_topic_preferences_invalid_config_fails_explainably` ? an invalid topic config causes `StageResult.fail`. This means the topic stage returns `passed=False` on **malformed config**.

Now trace this through the live wiring: if a user has a default policy whose `config_json` is somehow malformed (e.g., `{"topics":{"include":"AI"}}` ? a string instead of array, which F1's `/set` merge could theoretically allow, or a hand-edited/corrupted config), then `_evaluate_topic_preferences` returns a StageResult with `passed=False`. This turns a metadata-matched item into `matched=False` ? **silently dropped, never delivered**.

This is a real fail-closed-on-corruption concern. Let me verify with a repro.

43. assistant (2026-07-06T21:31:00.275Z)

```
import asyncio
import json
import aiosqlite
from datetime import UTC, datetime

from tg_video_filter import db
from tg_video_filter.models import Source, Policy, Account
from tg_video_filter.telethon_client import _evaluate_topic_preferences, _content_item_preview
from tg_video_filter.filters import VideoMeta


async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    acct = 111
    await db.upsert_account(conn, Account(id=acct, telegram_id=acct))
    src = await db.upsert_source(conn, Source(
        account_id=acct, provider="telegram", provider_source_id="5000",
        title="Cooking Channel", kind="channel", watched=True,
    ))
```

```
    now = datetime.now(UTC)
    meta = VideoMeta(size_bytes=100, duration_s=10, height=200, width=300,
mime_type="video/mp4")

    # Malformed topic config: include is a string, not an array.
    await db.upsert_policy(conn, Policy(
        account_id=acct, name="default",
        config_json='{"topics":{"include":"AI"}}',
        is_default=True, created_at=now, updated_at=now,
    ))
    item = _content_item_preview(acct, src, 1, 999, meta, "Best recipe ever", True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("malformed-config passed?", r.passed, "| reason:", r.reason)
    print("=> metadata-matched video would be", "DELIVERED" if r.passed else "DROPPED")

    # config_json that is not a JSON object at all (e.g. legacy flat or corrupt)
    await db.upsert_policy(conn, Policy(
        account_id=acct, name="default",
        config_json='"just a string"',
        is_default=True, created_at=now, updated_at=now,
    ))
    item = _content_item_preview(acct, src, 2, 998, meta, "recipe", True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("non-object-config passed?", r.passed, "| reason:", r.reason)

    await conn.close()


asyncio.run(main())
```

44. user (2026-07-06T21:31:00.276Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

45. assistant (2026-07-06T21:31:02.596Z)

```
A parallel session modified my repro.py. That's fine ? I'll write my second repro to a different
filename. Let me read then write.
```

46. user (2026-07-06T21:31:04.882Z)

```
1       import asyncio, aiosqlite
2       from tg_video_filter import db
3       from tg_video_filter.models import Source, User, Channel
4       from tg_video_filter.filters import VideoMeta
5       from tg_video_filter.telethon_client import _content_item_preview,
_evaluate_topic_preferences
6
7       USER_ID = 111
8       CHANNEL_ID = -1001234567890
9
10      async def main():
11          conn = await aiosqlite.connect(":memory:")
12          await db._apply_schema(conn)
13          await db.upsert_user(conn, User(id=USER_ID, phone="+15555550100"))
14
15          # Create source via upsert_channel (as the handler does)
16          channel = await db.upsert_channel(conn, Channel(user_id=USER_ID,
channel_id=CHANNEL_ID, title="AI Philosophy Daily"))
17          source = await db.get_source_by_id(conn, channel.id)
18          print("source:", source.id, source.title, source.provider,
source.provider_source_id)
19
20          # set a default policy with topics: include AI, exclude Crypto
```

```
21          await db.save_policy_config(conn, USER_ID, {"topics": {"include": ["AI"], "exclude":
["Crypto"], "mode": "any"}})
22
23          meta = VideoMeta(duration_s=100, width=1920, height=1080, size_bytes=5_000_000,
mime_type="video/mp4")
24
25          # Case A: caption mentions nothing, but title has "AI" -> include matches
26          item = _content_item_preview(USER_ID, source, 42, 999, meta, "some clip", True)
27          r = await _evaluate_topic_preferences(conn, USER_ID, source, item)
28          print("A include via title:", r.passed, r.reason)
29
30          # Case B: caption mentions Crypto -> exclude veto
31          item = _content_item_preview(USER_ID, source, 43, 1000, meta, "Crypto pump today",
True)
32          r = await _evaluate_topic_preferences(conn, USER_ID, source, item)
33          print("B exclude veto:", r.passed, r.reason)
34
35          # Case C: no default policy -> None
36          conn2 = await aiosqlite.connect(":memory:")
37          await db._apply_schema(conn2)
38          await db.upsert_user(conn2, User(id=USER_ID, phone="+1"))
39          ch2 = await db.upsert_channel(conn2, Channel(user_id=USER_ID, channel_id=CHANNEL_ID,
title="Neutral"))
40          src2 = await db.get_source_by_id(conn2, ch2.id)
41          item = _content_item_preview(USER_ID, src2, 44, 1001, meta, "hello", True)
42          r = await _evaluate_topic_preferences(conn2, USER_ID, src2, item)
43          print("C no policy:", r)
44
45          await conn.close(); await conn2.close()
46
47      asyncio.run(main())
48
```

47. assistant (2026-07-06T21:31:07.683Z)

```
Another session is running parallel repros. I'll use my own uniquely-named file to avoid
collision.
```

48. assistant (2026-07-06T21:31:17.081Z)

```
import asyncio
import aiosqlite
from datetime import UTC, datetime

from tg_video_filter import db
from tg_video_filter.models import Source, Policy, Account
from tg_video_filter.telethon_client import _evaluate_topic_preferences, _content_item_preview
from tg_video_filter.filters import VideoMeta


async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    acct = 222
    await db.upsert_account(conn, Account(id=acct, telegram_id=acct))
    src = await db.upsert_source(conn, Source(
        account_id=acct, provider="telegram", provider_source_id="5000",
        title="Cooking Channel", kind="channel", watched=True,
    ))
    now = datetime.now(UTC)
    meta = VideoMeta(size_bytes=100, duration_s=10, height=200, width=300,
mime_type="video/mp4")

    # Malformed topic config: include is a string, not an array.
    await db.upsert_policy(conn, Policy(
```

```
        account_id=acct, name="default",
        config_json='{"topics":{"include":"AI"}}',
        is_default=True, created_at=now, updated_at=now,
    ))
    item = _content_item_preview(acct, src, 1, 999, meta, "Best recipe ever", True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("malformed-config passed?", r.passed, "| reason:", r.reason)
    print("=> metadata-matched video would be", "DELIVERED" if r.passed else "DROPPED")

    # config_json that is not a JSON object at all
    await db.upsert_policy(conn, Policy(
        account_id=acct, name="default",
        config_json='"just a string"',
        is_default=True, created_at=now, updated_at=now,
    ))
    item = _content_item_preview(acct, src, 2, 998, meta, "recipe", True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("non-object-config passed?", r.passed, "| reason:", r.reason)

    await conn.close()


asyncio.run(main())
```

49. user (2026-07-06T21:31:17.137Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro_wiring_z.py
(file state is current in your context ? no need to Read it back)
```

50. user (2026-07-06T21:31:23.515Z)

```
malformed-config passed? False | reason: invalid topic config: topic include/exclude values must
be arrays
=> metadata-matched video would be DROPPED
non-object-config passed? False | reason: invalid topic config: policy config_json must decode
to a JSON object
```

51. assistant (2026-07-06T21:31:37.697Z)

Confirmed: a malformed default-policy `config_json` causes the topic stage to return `passed=False`, which turns any metadata-matched video into `matched=False` ? **silently dropped, never delivered** on both live and backfill paths. This is a fail-closed behavior on config corruption. Whether this is a "bug" depends on intent, but it's worth flagging as medium: a single malformed policy blob (e.g., a future `/set` writing a bad shape, or hand edit) silently suppresses ALL deliveries with no user-visible error (the reason is only logged in live, and not even logged in backfill).

Now let me check the biggest concern the prompt raised ? a divergence. Let me re-examine backfill more carefully for whether it appends the reason and whether it logs matched. Backfill does NOT log `video_seen` with reasons at all ? it just inserts. That's pre-existing though. Let me confirm the reason-append divergence is purely cosmetic (it is ? reasons only feed the log line in live).

Now, let me look at one more potential real issue: **the `_message_text` reading `message.message` vs `raw_text`**. On the backfill path, real Telethon Message objects have both. `message` is the raw text with entities stripped to plain? Actually `.message` is the raw text, `.raw_text` is also text. Fine. But note: for a video with NO caption, `.message` is `""` (empty string), which `_message_text` strips to `None`. Good.

Let me verify one concrete divergence concern the prompt flagged: **does backfill gate on topic result identically?** Yes (line 261 sets matched, line 278 gates delivery on matched).

Let me now double-check the `text` field flows correctly into `content_items.text` for dedup

lookups ? the reviewer verified this. Let me confirm there are no other call sites of VideoRecord construction that now need `text` but omit it (a regression where text is lost). Let me search.

52. user (2026-07-06T21:31:38.887Z)

```
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\backfill.py:263:                record =
VideoRecord(
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1571:    return VideoRecord(
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1730:    return VideoRecord(
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1814:        VideoRecord(
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\models.py:383:class VideoRecord(BaseModel):
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\telethon_client.py:293:        record =
VideoRecord(
```

53. assistant (2026-07-06T21:31:49.874Z)

All VideoRecord construction sites either set `text` (backfill:263, telethon:293, db shims) or default to None. `get_video_by_document_id` at db.py:1730 is `_content_item_to_video` which sets text. Good, no lost-text regression.

Now let me check `get_video` Tier-0 path ? it returns via `_content_item_to_video` which now includes text. Fine.

Let me now look at one important subtlety: **the exclude-only case and `enabled`**. If a policy has ONLY excludes (no includes), `enabled=True`. Then `matched_includes` isn't checked; `if not prefs.include:` returns pass. So exclude-only works as a pure veto. Confirmed by repro E earlier.

Now a genuinely important edge: **casefold substring matching false positives**. Topic "ai" would match "rain", "said", "email", "chair", etc. via substring. This is a design choice (documented as substring). Title "Rainbow AI" fine, but a topic like "art" matches "smart", "party", "start". This substring-not-word-boundary matching means excludes can over-veto and includes can over-match. Given the ADR describes "casefold substring match", this is intended per the PR, so at most a low/design-note. I'll note it as low since it can surprise users (an exclude of "art" silently drops "smart contracts" videos).

Let me verify substring behavior with a quick check and also confirm there are no crashes when title AND text are both None.

54. assistant (2026-07-06T21:32:00.033Z)

```
import asyncio
import aiosqlite
from datetime import UTC, datetime

from tg_video_filter import db
from tg_video_filter.models import Source, Policy, Account
from tg_video_filter.telethon_client import _evaluate_topic_preferences, _content_item_preview
from tg_video_filter.filters import VideoMeta


async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    acct = 333
    await db.upsert_account(conn, Account(id=acct, telegram_id=acct))
    # Title None to force reliance on caption only, and test both-None.
    src = await db.upsert_source(conn, Source(
        account_id=acct, provider="telegram", provider_source_id="5000",
        title=None, kind="channel", watched=True,
    ))
    now = datetime.now(UTC)
    meta = VideoMeta(size_bytes=100, duration_s=10, height=200, width=300,
```

```
mime_type="video/mp4")

    # substring false-positive: exclude "art" vetoes "smart contracts"
    await db.upsert_policy(conn, Policy(
        account_id=acct, name="default",
        config_json='{"topics":{"exclude":["art"]}}',
        is_default=True, created_at=now, updated_at=now,
    ))
    item = _content_item_preview(acct, src, 1, 999, meta, "smart contracts explained", True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("exclude 'art' vs 'smart contracts' passed?", r.passed, "reason:", r.reason)

    # both title and text None with include -> must not crash, should fail include
    await db.upsert_policy(conn, Policy(
        account_id=acct, name="default",
        config_json='{"topics":{"include":["ai"]}}',
        is_default=True, created_at=now, updated_at=now,
    ))
    item = _content_item_preview(acct, src, 2, 998, meta, None, True)
    r = await _evaluate_topic_preferences(conn, acct, src, item)
    print("both-None include passed?", r.passed, "reason:", r.reason)

    await conn.close()


asyncio.run(main())
```

55. user (2026-07-06T21:32:00.075Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro_substr_z.py
(file state is current in your context ? no need to Read it back)
```

56. user (2026-07-06T21:32:05.776Z)

```
exclude 'art' vs 'smart contracts' passed? False reason: excluded topic matched: art
both-None include passed? False reason: no include topics matched
```

57. assistant (2026-07-06T21:32:12.961Z)

```
Confirmed: substring "art" vetoes "smart contracts" (over-veto), and both-None doesn't crash.

Now, let me consider the **most severe potential issue** the prompt asked about: is there ANY
path where a topic-rejected item still gets delivered? Both handlers: `matched =
topic_result.passed`, then delivery gated on `matched`. No. And a topic-accepted item skipped?
Only if `matched` was already False from metadata ? but then topics don't run at all (`if
matched and source`). So topics can only demote. Correct AND semantics. No wrong-delivery bug
there.

Let me reconsider whether the fail-closed-on-malformed-config is genuinely reachable in
production, i.e., can F1's `/set` produce a malformed `topics` shape? Let me check how policy
config is written.
```

58. user (2026-07-06T21:32:14.425Z)

```
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:889:
config_json=row["config_json"],
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1261:
config_json=excluded.config_json,
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1295:def
_policy_config_json_to_dict(raw: str) -> dict[str, object]:
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1308:def
_policy_config_dict_to_json(config: dict[str, object]) -> str:
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1313:async def
```

```
load_policy_config(conn: aiosqlite.Connection, account_id: int) -> dict[str, object]:
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1314:    """Load the raw default
policy config as a JSON object.
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1326:async def save_policy_config(
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1329:    """Persist the raw default
policy config object."""
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1337:
config_json=_policy_config_dict_to_json(config),
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1793:    await
save_policy_config(conn, user_id, raw_config)
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1862:    "save_policy_config",
```

59. assistant (2026-07-06T21:32:21.546Z)

The malformed-config fail-closed is a latent/defensive concern rather than an easily-user-triggerable one via the current `/set` (which presumably validates), so I'll keep it medium. The reason it matters: the topic stage catches its own parse errors and returns fail (not raise), so a corrupted policy silently drops all matched videos rather than erroring loudly or failing open. And in **backfill it isn't even logged** (the reason isn't appended anywhere), so it's completely silent there.

Let me now finalize. Let me confirm the reason-not-logged-in-backfill divergence precisely ? backfill.py line 260-261 vs live 289-291. Backfill drops `topic_result.reason` entirely and has no `video_seen ... reasons=` log at all. Let me confirm backfill has no such log line.

60. user (2026-07-06T21:32:23.049Z)

No matches found

61. assistant (2026-07-06T21:32:33.845Z)

Confirmed. Backfill never logs the topic decision or reason ? a topic-demoted item in backfill is invisible (no log explaining why a metadata-matching video wasn't delivered). Live logs it via `reasons`. This is an observability divergence ? low/medium.

Let me do a final check on the `matched` stored on the throwaway ContentItem being passed as pre-topic `matched=True`. In `_content_item_preview`, `matched` is passed as the pre-topic value (always True at that call site since it's inside `if matched`). It's stored on the throwaway item but never read by the stage. Harmless, confirmed.

Now let me verify there isn't a subtle issue where `source.title` used in the live path could be stale/None because the item passed the `should_ingest_source` gate (which may have created a title-less row) but `upsert_channel` supplies the title from the event. On line 264 the source is re-fetched AFTER upsert_channel, so it gets the freshly-COALESCE'd title. Good. But if upsert_channel throws (line 265), source stays as the line-255 fetch, which for a watched channel has whatever title was previously stored. Acceptable.

I have enough. Let me clean up and report. The findings, ranked:

1. **Medium**: Malformed default-policy `config_json` fails closed ? silently drops ALL metadata-matched videos (both paths), and in backfill it's not even logged.
2. **Low**: Backfill drops `topic_result.reason` and has no decision log ? topic-demoted items are silently non-delivered with zero observability.
3. **Low**: Substring (non-word-boundary) topic matching over-vetoes/over-matches (e.g., exclude "art" drops "smart contracts").

Let me clean up my scratchpad repros.

62. user (2026-07-06T21:32:37.750Z)

done

63. assistant (2026-07-06T21:32:40.142Z)

I've completed the wiring review. My findings, ranked most-severe first.

64. user (2026-07-06T21:33:07.412Z)

Structured output provided successfully